

Description

The directory containing icons or images is called *images*, which we will be using in our extension.

manifest.json is a metadata file in JSON format that contains basic properties like the extension's name, description, version number, etc.

background.js is a JavaScript file containing the search logic for the extension.

Adding icons or images to our extension

In this step, we are going to add an icon to our option in the context menu. Visit <http://www.flaticon.com/search/magnifier> and download any one icon, preferably in .png format, and of size 16x16 pixels. Copy it to the *images* folder located in our extension's folder and rename that image to 'search.png'.

Creating the manifest file

Open the *manifest.json* file with your favourite code editor or Notepad and copy the following JSON code:

```
{
  "manifest_version": 2,
  "name": "Privacy Search",
  "description": "This add-on searches highlighted text, on
selected top three privacy search websites.",
  "version": "1.0",
  "background": {"scripts": ["background.js"]},
  "icons": { "16": "images/search.png"},
  "permissions": ["contextMenus", "tabs"]
}
```

The code is written in JSON format (Key: Value pair). It consists of all basic information about the extension such as its name, description, version, images, icons used, permissions required and so on.

Parameter description

- 'manifest_version' specifies the version of the manifest; the value must be 2.
- 'name' specifies the name of the extension.
- 'description' specifies a short description of the extension.
- 'version' specifies the version of the extension.
- 'background' specifies that script 'background.js' loads as soon as the user starts the Opera browser.
- 'icons' specifies the location of the icons or images, which we are going to use in our extension.
- 'permission' specifies which Opera extension APIs are to be used. We are using *contextMenus* and *tabs*.

Adding JavaScript

Now, open the *background.js* file in your favourite code editor

or Notepad and copy the following JavaScript code:

```
var parent = chrome.contextMenus.create({
  title: "Privacy Search on..",
  contexts: ["selection"]
});

chrome.contextMenus.create({
  title: "Search on DuckDuckGo for: \"%s\"",
  parentId: parent,
  contexts: ["selection"],
  onclick: searchTextDuckDuckGo
});

chrome.contextMenus.create({
  title: "Search on Encrypted Google for: \"%s\"",
  parentId: parent,
  contexts: ["selection"],
  onclick: searchTextGoogle
});

chrome.contextMenus.create({
  title: "Search on Ixquick for: \"%s\"",
  parentId: parent,
  contexts: ["selection"],
  onclick: searchTextIxquick
});

function searchTextDuckDuckGo(info){
  var myQuery = encodeURIComponent("https://duckduckgo.
com/?q="+info.selectionText);
  chrome.tabs.create({url: myQuery});
}

function searchTextGoogle(info){
  var myQuery = encodeURIComponent("https://encrypted.google.
com/#q="+info.selectionText);
  chrome.tabs.create({url: myQuery});
}

function searchTextIxquick(info){
  var myQuery = encodeURIComponent("https://ixquick.com/do/
search?q="+info.selectionText);
  chrome.tabs.create({url: myQuery});
}
```

The first four functions are for creating a UI for our extension. Since we are providing options in the context menu, we will use *contextMenus.create()* function to create a customised context menu. This function has some key:value parameters such as title and contexts; the most important one is the event – which specifies what must be done when certain browser based events occur, i.e., calling